ENDOR LABS

Malware in Open Source Ecosystems

Everyone's Problem, No One's Program

March 2026

Contents

3 **Executive Summary**

4 **Introduction**

5 **The Threat Is Real, and It's Already on Your Dev Machines**
- The volume
- Account takeovers
- The malware attack window
- What this means for your program

7 **Concern Is Rising — But So Is the Gap**
- Priority without investment
- Half of respondents had a malware incident
- Where detection happened
- What this means for your program

9 **The Attack Surface Is Bigger Than Most Realize**
- Package upgrade frequency
- Centralized visibility
- Ecosystem risk
- Transitive dependencies
- What this means for your program

11 **The Controls Are There. The Coverage Isn't.**
- Artifact repositories and registry proxies
- Versioning and lockfiles
- The knowing-doing gap
- The speed-security tradeoff
- What this means for your program

13 **npm Controls Exist But Aren't Widely Adopted**
- What the Endor Labs npm metadata analysis shows
- The Shai-Hulud data
- Why npm Trusted Publishing adoption is low
- The Consumer Gap
- What this means for your program

15 **Responsibility Is Everywhere. Accountability Is Nowhere.**
- How ownership is structured
- Personal visibility into controls
- Detection fragmentation
- How organizations learn about new malware
- Response times
- What this means for your program

17 **The Malware Task Force Imperative**
- Who needs to be in the room
- What the program needs to address
- Putting it into practice

Executive Summary

For most of the history of open source software (OSS), organizations centered their threat models on vulnerabilities: known weaknesses in legitimate code that needed to be patched. That model is no longer complete.

Malware is present in many different types of open source components. This report is narrowly focused on a large category: software dependencies used in application code.

Malicious open source packages — code that was never trustworthy to begin with, or packages that were taken over by bad actors — have emerged as a distinct and fast-moving category of risk. This type of risk hit mainstream awareness with the 2024 XZ Utils compromise and escalated in 2025 with the high-profile Shai-Hulud campaign.

This report draws on two sources: a survey of 605 IT professionals across the North America, Europe, and India, conducted January–February 2026; and original research from Endor Labs analyzing malware advisories in the publicly available Open Source Vulnerability (OSV) database and npm package metadata.

The threat is growing fast

~14x
more OSV malware advisories in the last two years

90%+
of all OSV malware advisories ever filed came in 2025

92%
of all npm account takeover advisories were filed in 2025

Organizations aren't keeping up

51%
identified suspected or confirmed malicious packages in 2025

81%
say it's a top priority
only 48% expect budget to increase

Priority without investment

88%
know new releases are highest-risk
only 21% enforce a cooldown period

Awareness without action

This report documents the gaps, why they persist, and what it takes to close them.

Introduction

It is estimated that 70% to 90% of a modern application's codebase is composed of OSS, rather than code written by an organization's software developers. The average enterprise application pulls in hundreds of direct dependencies, each of which may call dozens of transitive ones — a chain that grows fast and changes constantly. For most of open source security's history, that chain's primary risk was vulnerabilities: known weaknesses in legitimate code that attackers could exploit if left unpatched.

That's still true. But it's no longer the whole picture.

Malware in open source packages operates differently. While vulnerabilities are code bugs that could hypothetically be exploited, malware is code designed to cause harm. And with the former, generally danger is only present if the vulnerability makes it into production. But with open-source malware, the attack can start the second the package is imported. The detection surface is different. The response playbook is different.

This report is built on two complementary sources of evidence. The first is a survey of 605 IT professionals — spanning engineering and security roles — conducted January 26–February 10, 2026, across North America, Europe, and India. The second data stems from original research by Endor Labs. The security research team analyzed malware advisory data from OSV and meta data from npm, with a focus on account takeover trends and security control adoption among npm package maintainers.

Both point to the same conclusion: the threat is real, concern is rising, and organizational defenses have a lot of catching up to do.

The Threat Is Real, and It's Already on Your Dev Machines

2025 was a loud year for open source malware — louder, in some ways, than the raw numbers alone can explain. Better detection plays a role: more researchers looking, more tooling in place, more advisories filed on packages that had been quietly malicious. But the volume is also real. Both things are true, and supply chain security strategies will be shaped by both.

The volume

OSV has accepted malware advisories since the database was established in 2021. However, total malware advisories stayed relatively small until 2024. Reports increased modestly through 2024, then exploded in 2025. We went from 9,721 reports at the beginning of 2024 to over 200k by the end of 2025. That's a 1,267.5% increase within a 24-month period.

The threat is concentrated in two ecosystems: npm (JavaScript/TypeScript) and PyPI (Python), which together account for the vast majority of all OSV malware advisories. Of all OSV malware advisories across those two ecosystems, over 90% — more than 192,000 — were reported in 2025 alone. Within that pair, npm has been the dominant target: 191,418 advisories in 2025, compared to 8,371 in 2024. That's a more than 20x increase in a single year.

A significant portion of that increase can be traced to the IndonesianFoods campaign, which generated substantial advisory volume on its own. But the broader pattern — more malicious packages, more campaigns, more ecosystems targeted — holds regardless of any single event. npm's architecture and its historically permissive security model, combined with the pervasiveness of JavaScript and its dependency radius, make it an attractive target. More people are looking for malware in npm, and there's more malware to find.

Application security professionals have been incentivized by regulations, standards, and customers to focus on vulnerability management — CVEs, patch cycles, dependency audits. Malware in the supply chain is a different animal.

Account takeovers

One category within the malware surge deserves specific attention: account takeovers, or ATOs. An ATO is when an attacker gains access to a legitimate package and publishes a malicious new version containing malware. The package name doesn't change. Even the maintainer identity might stay the same. What changes is the version number and the code.

Of the 1,011 npm ATO advisories recorded in OSV across all years, 930 were filed in 2025 — roughly a 12x year-over-year increase. Several high-profile campaigns contributed to that spike, most notably Shai-Hulud.

> **"In my opinion the most concerning scenario is compromised maintainer accounts on high traffic trusted package because they allow attackers to inject malicious code directly into legitimate updates. This creates a massive blast radius where automated CI/CD pipelines unknowingly distribute infostealers to thousands of downstream environments."**
>
> *– Senior Manager, AppSec, India*

ATOs are a particularly dangerous form of malware because the attackers take advantage of trust built by the real maintainers. In addition, the engineering best practice of "always upgrade to latest" — used to remediate vulnerabilities — ensures people will accidentally consume malicious versions in their effort to follow security standards and customer expectations. Threat actors often target popular packages to ensure maximum payoff.

Among the 2025 npm ATO cases:

38.4%
of affected packages had more than 1,000 monthly downloads

18.5%
exceeded 10,000

11.1%
had more than 100,000

Attackers aren't going after the margins — they're going after packages that are deeply embedded in production systems.

The malware attack window

When a malicious version is published, every pipeline configured to update automatically is a potential victim — within seconds of publication, before any human review. Threat actors know that the malware will likely be removed from npm within days, so your organization's risk window is determined by how quickly your systems pull new versions. In documented ATO cases, malicious versions that existed for just a few hours were still automatically pulled into thousands of pipelines before anyone caught them.

> **[I worry about] Leak of secrets which will lead to bigger compromise in the future. For example TruffleHog deployed as part of the Shai-Hulud compromise was a real headache — we had to rotate a lot of secrets. From SecOps perspective and knowing that compromised credentials/secrets are one of the most common initial breach vector, this makes me and my team paranoid."**
>
> *– IC, Security Operations/Detection & Response/Incident Response, Scotland*

After a malicious version is identified, removal isn't guaranteed. Endor Labs' research revealed that more than 63,000 malicious packages remain in npm, despite having corresponding OSV malware advisories. The vast majority were published by bad actors from the start — no legitimate maintainer to pull them, no internal incentive to act. These packages have been reported to both OSV and npm. OSV issues the advisory; npm is responsible for the actual removal. For whatever reason, npm hasn't taken them down. For ATO-related packages, the picture is different: when legitimate maintainers discover a compromise, they act. Of the 930 ATO packages reported in 2025, roughly 2.5% remain unremoved. For non-ATO malicious packages, that non-removal rate is closer to 32%.

What this means for your program: The threat isn't theoretical, and it isn't evenly distributed. npm is a major exposure surface. Auto-updating workflows create the shortest path from publication to impact. And a significant volume of known-malicious packages are still downloadable from npm. Dependency intake controls — where packages come from, how they're vetted, and how updates are managed — are the first line of defense.

Concern Is Rising — But So Is the Gap

Practitioners are paying attention. What's equally clear is that awareness hasn't translated into proportional action.

Priority without investment

81%
of respondents say malicious OSS is a top security priority for their organization in 2026.

60%
say their concern has increased — and not a single respondent said their concern decreased significantly.

Only 48%
expect their organization's spending on malicious OSS prevention and detection to increase in 2026.

That means roughly **one in five** said concern increased but investment isn't expected to follow. Concern and action are moving at very different speeds — and that gap has real consequences given **51%** experienced a malware incident in 2025.

> **"I'm not worried [about malicious dependencies], everybody knows we're vulnerable to that and nobody wants to fix it, because it's running fine so why bother."**
>
> *– Manager, Software Engineering, Germany*

Who's falling behind?

The gap between concern and investment is the overall picture, but for several industries, it's far worse. Media and Entertainment respondents showed a 33-point gap between rising concern and rising investment. Energy and Utilities weren't far behind at 29 points. Retail and eCommerce at 26. These aren't low-risk sectors, but they are sectors that may be underestimating their exposure. Smaller companies (under 500 employees) showed the widest size-based gap at 17 points, precisely where program maturity and dedicated security resources tend to be thinnest.

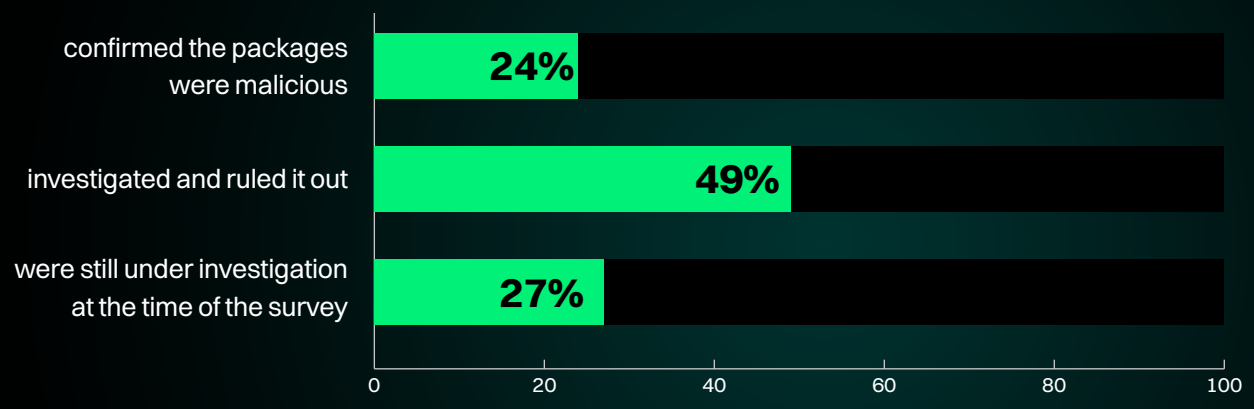
That gap carries compounding risk. Without dedicated security headcount, consistent tooling, or formal incident response processes, the distance between "we know this is a problem" and "we've done something about it" is harder to close. For organizations living below the "security poverty line," the consequences of a malicious package exfiltrating secrets can be much more significant. Smaller companies are also more likely to rely on a single engineer or small team making ad hoc dependency decisions, without the centralized visibility or enforcement infrastructure that larger organizations have had years to build.

In other words, concern is not a substitute for a program. And for the organizations with the fewest resources to absorb a supply chain incident, the gap is exactly where threat actors look for an opening.

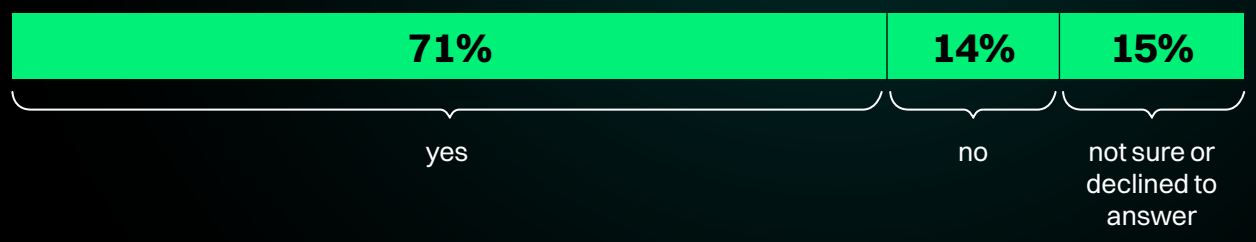
Half of respondents had a malware incident

51% of respondents said their organization identified suspected or confirmed malicious OSS packages in 2025. That reframes the concern-vs-budget conversation entirely. For more than half of the people in this survey, the threat already arrived. The question for the other **49%** is whether they're well-defended, have less surface area, or simply don't know it happened.

Of those who experienced a malware incident:



Triggered a formal incident response (IR) process?



Where detection happened

Among those who identified malicious packages, they were discovered in several places: developer machines (30%), CI/build pipelines (20%), production environments (20%), artifact repository or proxy layers (19%). No single detection layer is catching everything — organizations relying on only one checkpoint are leaving gaps. However, one malware fact should be remembered: The developer is usually patient zero. So even in cases where malware was found elsewhere, it likely passed through a developer machine.

What this means for your program: The concern-investment gap is worth surfacing with leadership. The 51% who encountered malicious packages make the case: this isn't a future risk to prepare for, it's a current one to manage.

The Attack Surface Is Bigger Than Most Realize

Exposure to malicious OSS scales with how frequently and broadly dependencies move through an organization's systems. For most organizations, that movement is constant.

Package upgrade frequency

60% of respondents say developers in their organization upgrade open source packages at least weekly. **8%** do it multiple times a day. While upgrades are crucial to remediating vulnerabilities and decreasing tech debt, they're also a potential malware exposure point. Not because most packages are malicious, but because the ATOs are designed to blend into normal upgrade activity.

38%

Teams that didn't identify malicious OSS in 2025 were **38%** more likely to say upgrades happen monthly or less. That's worth reading carefully — it probably doesn't mean those teams are better protected.

Lower upgrade frequency tends to correlate with industries where software development moves more slowly, and with organizations that have less surface area or less visibility into what's happening in their pipelines. Dramatically slowing down upgrades isn't the answer, and it creates its own risk by leaving legitimate security patches unapplied. The takeaway is that controls need to operate at the same cadence as the dependency changes they're supposed to govern. (Exception: Cooldown policies. More on that later.)

Centralized visibility

24% of respondents say their organization has little to no centralized visibility over where dependencies come from or how and when they change. Controls on paper don't help much if nobody has a clear picture of where dependencies actually originate or when they change — enforcement becomes whatever the last team remembered to do.

The link between visibility and prioritization runs both directions: those who said malicious OSS was not a top priority were **78%** more likely to report limited visibility. Organizations that aren't looking for a threat tend not to see it, making the threat appear smaller and easier to deprioritize.

Ecosystem risk

npm users were **19%** more likely than average to report confirmed or suspected malicious OSS still under investigation at the time of the survey — a direct reflection of the advisory concentration in the npm ecosystem documented in Section 2. Organizations with heavy JavaScript/TypeScript footprints are operating with structurally higher exposure.

> **The scenario that worries me the most is a quiet, widely-used transitive dependency getting compromised and exfiltrating secrets at build or runtime and doing it just subtly enough that it blends into normal behavior."**
>
> *– Senior Manager, Software Engineering, The Netherlands*

> **We're most concerned about a compromised transitive dependency being introduced through a routine update and propagating through CI before it's detected, especially if it's widely used and difficult to replace quickly."**
>
> *– IC, DevOps/Platform Engineering, United States*

Transitive dependencies

Most dependencies in production software aren't ones anyone explicitly chose — they're pulled in by the packages you did choose, which pull in more, and so on. Even organizations with rigorous review processes for direct dependencies are rarely vetting the transitives. That's not a process gap so much as a structural reality of how modern software is assembled. But it means the actual dependency surface is almost always larger than it appears.

What this means for your program: Visibility into what's being pulled, from where, and how often is a prerequisite for everything else. Without it, controls are incomplete and the gaps are invisible.

The Controls Are There. The Coverage Isn't.

Most organizations have at least some dependency controls in place. What varies, sometimes dramatically, is how consistently they're applied and how much of the actual dependency surface they cover. Different organizations take different approaches, and that's not inherently wrong. What the data shows is that gaps in coverage tend to be invisible until something goes wrong.

Artifact repositories and registry proxies

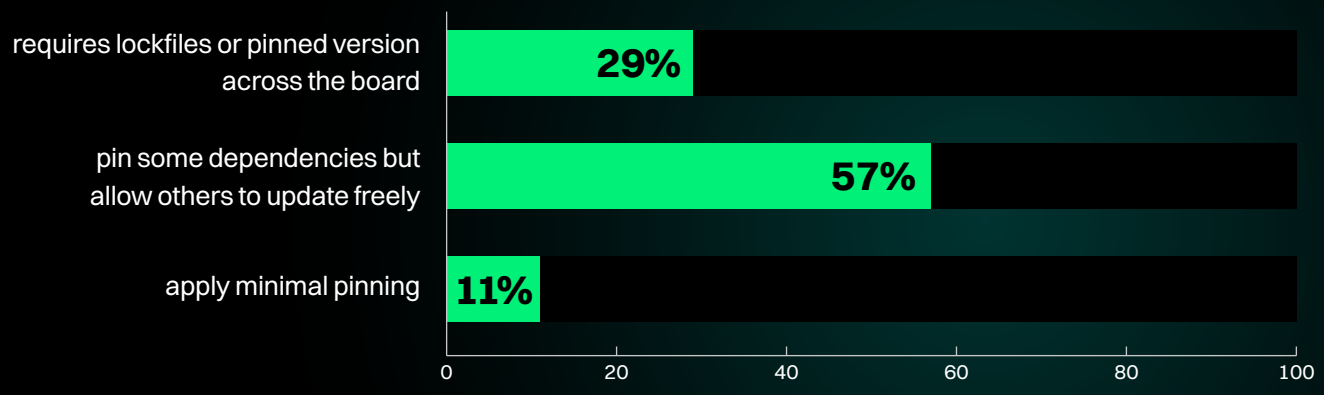
43% of respondents say most of their dependency downloads flow through an internal artifact repository or registry proxy. Those who do this consistently were **29%** less likely to have identified malicious packages in 2025 — one of the cleaner cause-and-effect relationships in the data.

17% have no dependency management controls, and **40%** are partially managing dependency consumption. In practice, partial often means governing some applications while others manage their own dependencies directly — one team locked down, another pulling straight from public repos.

Versioning and lockfiles

Lockfiles and version pinning are standard engineering hygiene, but they also matter specifically for malware prevention. When dependencies are free to update automatically, a malicious version published to a trusted package can be pulled into a pipeline without any human decision being made. Pinning doesn't eliminate that risk, but it adds a deliberate step. Someone has to actively choose to upgrade, which creates an opportunity to catch something before it lands. The Endor Labs blog on defending against npm supply chain attacks goes deeper on why these practices matter beyond just stability.

Does the organization require lockfiles or pinned versions?



The knowing-doing gap

88%
of respondents agree that the first days after a new package release are the highest-risk window for consuming malicious packages.

21%
enforce a cooldown period before adopting newly released versions.

That's one of the sharpest disconnects in the survey. The awareness is there. The practice isn't. Whether cooldown periods are hard to implement at scale, create too much friction in fast-moving development cycles, or just haven't been formally prioritized is something the data alone can't answer, but the gap is real enough to call out. The usefulness of cooldown policies, and how to implement them, is covered in another Endor Labs blog: Why Cooldown Windows Belong in Every npm Security Strategy.

The speed-security tradeoff

Of those with at least one dependency guardrail in place, **47%** say enforcement varies by team or repository; **5%** say exceptions or bypasses are frequent. Every security program needs an exceptions policy — guardrails that can't flex tend to break entirely, and teams will always find ways around controls that block them from shipping. The question isn't whether exceptions exist. It's whether they're tracked, time-limited, and followed by remediation.

The most common reasons guardrails give way: exceptions granted to keep work moving **(30%)**, teams routing around controls through direct upgrades or local overrides **(28%)**, legacy tools that don't support enforcement **(23%)**, and policies that only exist in some repos or teams **(18%)**. The legacy tool challenge skews toward larger organizations — enterprises with 5,000 or more employees are disproportionately affected, where technical debt accumulates across more teams and longer timelines.

> **[I'm worried about] Replaced dependencies implementing breaking changes which then get pushed to production or take days or weeks to be fixed without a real value being generated, which unfortunately is one of the KPIs we are rated by."**
>
> *– Manager, Software Engineering, Germany*

The **28%** who admit to bypassing controls is almost certainly an undercount. Self-reported non-compliance in anonymous surveys runs conservative. Bypassing a control to keep a build moving is a routine operational call, not something most people register as a policy violation. The real number is probably higher, and the more useful framing is that bypasses happen in nearly every organization, because the alternative is grinding delivery to a halt.

What this means for your program: Coverage matters as much as existence. An artifact proxy that governs **60%** of dependency traffic and a lockfile requirement that **40%** of teams work around are less protective than they look on paper. The starting point is understanding what's actually being enforced, not what the policy says.

npm controls exist but aren't widely adopted

This is where the two sides of the research converge. The survey shows that organizations aren't consistently protecting their dependency intake. npm data shows that maintainers aren't adopting all available security measures. The barriers are different on each side, but the result is the same: the tools exist, and most people aren't using them.

What the Endor Labs npm metadata analysis shows

Endor Labs' visibility into npm maintainer security is limited to what's publicly available in package metadata — internal controls like MFA aren't visible, and that's appropriate given the security implications of that kind of transparency. What metadata does reveal is whether maintainers are using attestations or Trusted Publishing, two controls that communicate when a package was published by the genuine package maintainer.

In July 2025, npm launched Trusted Publishing — a method for publishing packages from CI/CD workflows using OpenID Connect (OIDC) authentication, reducing the need for long-lived npm tokens (a primary vector for account takeovers). Combined with npm attestations — cryptographically verifiable records of where and how a package was built — Trusted Publishing is what a lot of the security community is pointing to as the gold standard for npm package security.

Of the 930 npm packages confirmed as ATO victims in 2025, just over **14%** are currently using at least one of these controls. That number matters because of the population it describes. These are packages that were already compromised. If getting taken over isn't enough to prompt adoption of controls designed specifically to prevent account takeovers, the barriers are bigger than awareness.

The Shai-Hulud data

The Shai-Hulud campaign offers a more granular view. Of the 808 packages affected, **2.5%** were using either attestations or Trusted Publishing before the first wave hit in September. Small adoption spikes followed each wave: roughly **5%** of first-wave packages added controls afterward; roughly **9%** from the second wave did. Today that percentage is at 1**0.8%** — real movement from the starting numbers, but modest given what those maintainers had just been through, and below average when compared with all victims of ATOs. This illustrates that high-profile incidents create some movement. They don't fundamentally shift adoption rates.

Why npm Trusted Publishing adoption is low

Trusted Publishing is a concept in many ecosystems. The npm implementation has real infrastructure constraints. It currently works with GitHub Actions and GitLab CI/CD. Maintainers using other CI systems can't adopt it without migrating their build infrastructure — a significant ask for organizations, nevermind individual open source maintainers who don't have a DevOps team behind them. Setting up OIDC tokens and configuring runners correctly can be a genuine technical lift.

There's also a design problem. Even after a maintainer enables Trusted Publishing, the default still allows packages to be published through other methods, including tokens that ATOs exploit. Enforcing Trusted Publishing as the sole publishing path requires a second, non-obvious step, which means many packages have Trusted Publishing enabled but not enforced.

For context: Trusted Publishing isn't a new concept. PyPI has had it much longer than npm, and adoption rates in that ecosystem appear meaningfully higher. PyPI is still the second-largest source of malicious packages — but a distant second to npm. The gap suggests the problem isn't the concept, it's npm's late implementation and immaturity.

The consumer gap

Even if maintainer adoption were high, consumers face their own structural problem. Today's tooling doesn't surface package metadata — including whether a package uses Trusted Publishing or has attestations — at the point of consumption. The information exists, but it's not typically part of upgrade workflows. Checking requires consumers to actively review metadata for every package they pull, before every install.

For transitive dependencies, that's not a realistic ask. The volume is too large and the tooling to make it manageable doesn't broadly exist. The current ecosystem places an unrealistic burden on the consumer: vet every direct dependency and every transitive one, every time. Few organizations are strictly vetting all dependencies. While it's possible with tools like Endor Labs, the reality is a major process change that most engineering teams aren't willing to accept. Yet again, we're left with a speed-security tradeoff.

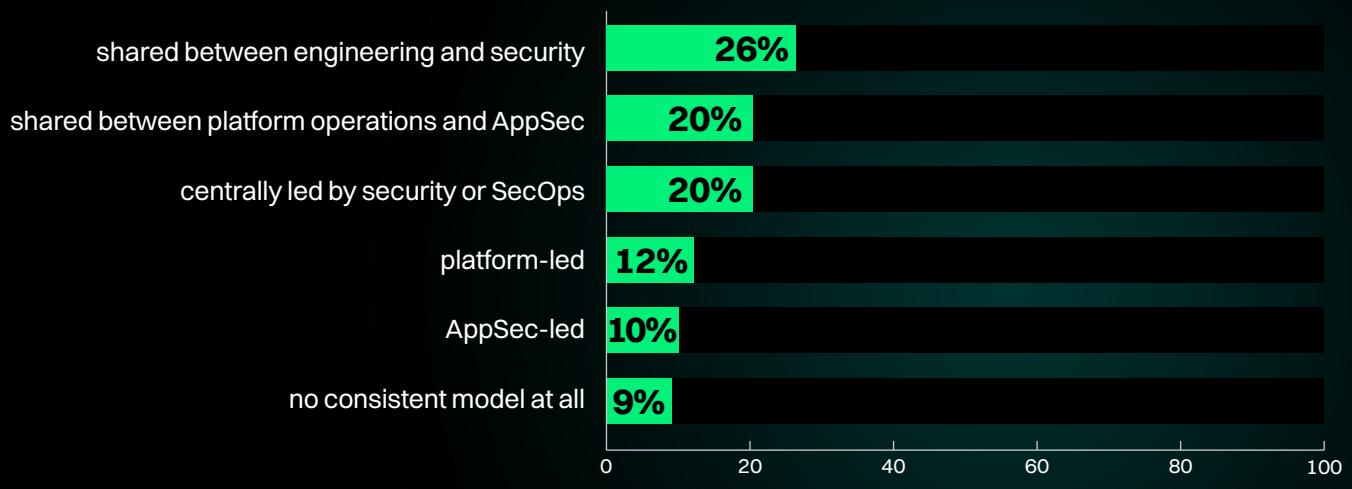
What this means for your program: Trust in the registry is not a control. Packages that look legitimate can be compromised, and npm's moderation process is not real-time. Design your security program with that assumption built in — not as a worst case, but as the baseline.

Responsibility Is Everywhere. Accountability Is Nowhere.

Even in organizations where controls exist, a consistent structural problem undermines them: nobody fully owns this.

How ownership is structured

The survey shows five distinct ownership models in roughly equal distribution:



None of these structures is inherently wrong. What the distribution reflects is an industry still figuring out where malicious OSS fits in the org chart. When you look at the full list of controls needed to prevent malware from entering and persisting in a software supply chain — dependency intake, scanning, artifact management, detection, incident response — it spans at least four distinct functions. No single team owns all of it, which means coordination isn't optional. It's the whole game.

Personal visibility into controls

41% of individual respondents say they have little to no visibility into their organization's dependency intake controls — they don't know where scanning happens, which guardrails are enforced, or what would flag a problem.

100%

The role breakdown is where this gets operationally significant. Respondents in Security Operations, Detection & Response, and Incident Response roles — the people who get called when a malicious package is identified — were **100%** more likely than average to report having no visibility into dependency intake controls. They're being handed a crisis with a map that has most of the landmarks missing. They don't know what controls were supposed to catch it, what failed, or where to start looking.

Detection fragmentation

14% of respondents say detection varies by team or happens mostly after an alert or incident has already been flagged. Combined with the visibility gap, that's a significant portion of organizations where detection is neither consistent nor proactive.

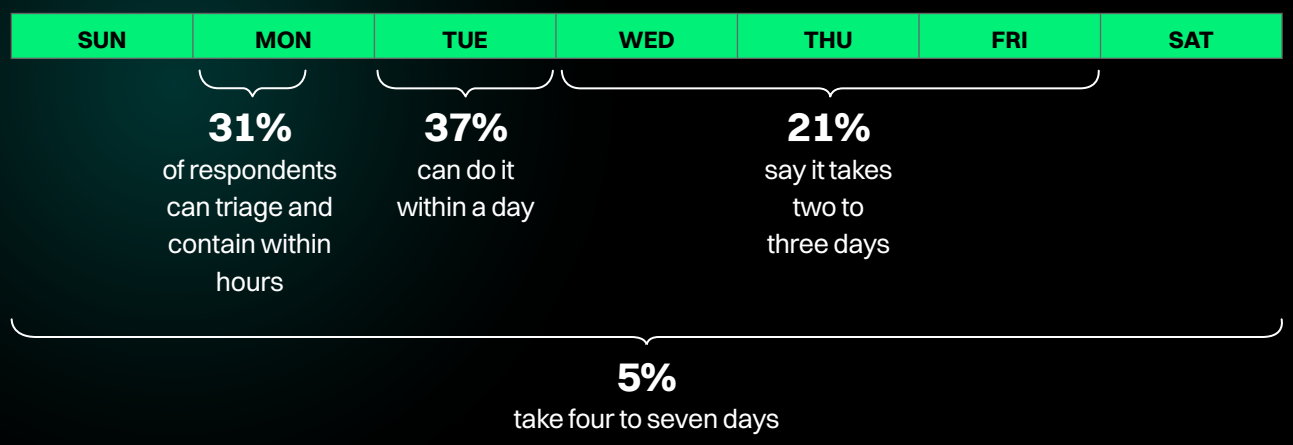
How organizations learn about new malware

83% of respondents rely on at least one external source to learn about newly discovered malicious OSS — scanner alerts, vendor notifications, public advisories, security news. That's not a red flag on its own. External tools that surface a malicious package before it's installed can function as genuinely proactive controls. Not every organization can staff a dedicated cyber threat intelligence function — that's largely an enterprise capability — and well-integrated external feeds can meaningfully close the gap.

What matters is whether those signals reach the right people in time to act, and whether a process exists to act on them when they do. Threat intelligence without dependency visibility is just noise. If you can't immediately cross-reference a new advisory against what's actually running in your environment, a day or more can pass before you know whether you're affected. An always-current SBOM turns that question from a days-long investigation into a seconds-long lookup.

Response times

Response time when an organization confirms they're using suspicious packages or confirmed malware:



The **28%** who take more than a day face a specific challenge: once malware is in your environment, the package getting taken down doesn't help you. The exposure happened at download. Dwell time after install is what determines how much damage it can do.

Top barriers to faster response: limited staffing and time **(21%)**, dependencies that are deeply embedded and hard to replace **(17%)**, unclear ownership and team handoffs **(15%)**, too many alerts making it hard to identify true positives **(15%).**

What this means for your program: If the team responsible for responding to a malicious package incident doesn't know where your controls live, your incident response plan has a gap. Visibility into the dependency intake program needs to extend beyond the people who built it.

The Malware Task Force Imperative

The survey and Endor Labs' research point to the same structural gap: malicious OSS is being managed as someone's problem rather than as a shared program. The controls exist in pieces. The awareness is there. What's missing is the coordination.

Closing that gap doesn't require a new category of tooling. It requires treating malicious OSS as a distinct threat category with its own cross-functional program, its own governance, and its own response playbook.

Who needs to be in the room

Four functions need coordinated ownership over this problem — and each has a distinct role:

- **Engineering** owns safe dependency consumption. Lockfiles, minimizing dependency sprawl, avoiding blind auto-upgrades, MFA on npm accounts. These aren't exotic controls. They're the baseline, and the survey shows they're inconsistently applied.
- **Application Security** owns pipeline hardening and guardrail enforcement. Integrity checks, pinned versions in CI, install script controls, cooldown policies, malware detection integrated from AI IDE through production. AppSec sets the rules; the program makes sure they're actually followed.
- **Cloud Security** owns blast containment and infrastructure resilience. Network segmentation, least-privilege access, runtime protection, key management. When a malicious package does get through, cloud security determines how far it can travel. And given that many malware campaigns seek to exfiltrate secrets, cloud security must be prepared for stolen cloud credentials.
- **Security Operations and Incident Response** owns early detection and rapid counter-response. Threat intelligence feeds, IR playbooks specifically built for supply chain attacks, threat hunting for indicators of compromise. These are the people who get called when something lands — and the survey shows they often don't know where the controls live.

What the program needs to address

Risk tolerance for OSS consumption. Not all dependencies carry equal risk. Packages from high-volume ecosystems like npm, newly released versions, and packages with large footprints but few active maintainers warrant closer attention. The program should define what acceptable risk looks like and what triggers additional review.

Dependency intake controls — and consistent enforcement. Artifact proxies, lockfiles, and allowlists work when they're applied. The program should define where controls are required, how exceptions are approved and tracked, and how compliance is monitored — not just assumed.

Cooldown periods and version pinning. **88%** of practitioners know the first days after a release are highest-risk. **21%** have built a practice around that knowledge. Knowing the risk and still not acting is a choice — and it's one the program should force into the open.

Detection coverage across the full pipeline. Developer machines, CI/CD pipelines, artifact repositories, production environments. The survey shows malware turning up at every stage. Detection at only one layer isn't a program, it's a checkpoint.

A response playbook. **14%** of organizations that found malicious packages in 2025 didn't run a formal IR process. A playbook answers the questions that can't wait until the middle of an incident: who gets called, what gets isolated first, how credentials are rotated, how the dependency chain is traced back to the source.

Putting it into practice

A cross-functional program isn't just an org chart change — it requires the teams to actually work together. Three exercises help make that real:

Threat modeling brings all four functions together to identify the attack paths most likely to hit your environment before they do. Developers map critical dependencies, AppSec sets cooldown policies, cloud security identifies lateral movement risks, and SecOps builds playbooks around the modeled scenarios.

Purple teaming validates that defenses actually hold. Using a real campaign like Shai-Hulud as the scenario, offensive and defensive teams work together in real time — the attacker explains the technique, the defenders share what their tooling actually caught. Theoretical knowledge becomes tested knowledge.

Tabletop exercises test whether people and policies are ready. No live malware — just a facilitator, a scenario, and the question of whether the organization responds with organized speed or improvised panic.

ENDOR LABS

Authors

Henrik Plate, Head of Security Research, Endor Labs
Luca Compagna, Security Researcher, Endor Labs

About Endor Labs

Endor Labs is the AI-native application security platform for teams that refuse to compromise between speed and security. Endor Labs helps teams identify, prioritize, and fix vulnerabilities across code, open-source dependencies, and container images. With agentic reasoning, deep program analysis, automated remediation, and unmatched coverage, Endor Labs empowers modern engineering and security teams to move fast without compromise.

Survey Methodology and Demographics

The Malware Measured survey was conducted between January 26 and February 10, 2026. Kickstand surveyed a total of 605 IT professionals employed full-time in roles spanning DevOps/Platform Engineering, Site Reliability Engineering, DevSecOps, Security Engineering, Application Security, Product Security, Software Engineering (manager level and above), and Security Operations/Detection & Response/Incident Response. Respondents were based in the North America, Europe, and India, and worked at organizations with 100 or more employees across technology, financial services, healthcare, manufacturing, telecommunications, retail, energy, and media industries. The study was conducted at 95% confidence with a +/- 4% margin of error.

© 2026 Endor Labs. All rights reserved.